

SOFT COMPUTING

ASSIGNMENT -11

Perumalla Dharan

AP21110010201

```
import numpy as np

from tensorflow.keras.datasets import mnist

# Loading and normalizing MNIST Dataset

def fetch_mnist():

    (train_X, train_y), (test_X, test_y) = mnist.load_data()

    # Scaling pixel values to the range [0, 1]

    train_X = train_X.astype(np.float32) / 255.0

    test_X = test_X.astype(np.float32) / 255.0

    return (train_X, train_y), (test_X, test_y)

# ReLU activation function for handling non-linearity

def relu_activation(matrix):

    return np.where(matrix > 0, matrix, 0)

# Softmax activation for class probabilities

def softmax_activation(matrix):
```

```

    shift = matrix - np.max(matrix)

    exps = np.exp(shift)

    return exps / np.sum(exps)

# Performing Convolution operation

def convolution(input_mat, filter_mat, stride=1, pad=0):

    # Adding padding to input

    input_padded = np.pad(input_mat, ((pad, pad), (pad, pad)),
mode='constant')

    input_size, filter_size = input_padded.shape[0],
filter_mat.shape[0]

    output_dim = (input_size - filter_size) // stride + 1

    conv_output = np.zeros((output_dim, output_dim))

    for x in range(0, output_dim):

        for y in range(0, output_dim):

            # Selecting a region of the image that matches the size of
filter

            region = input_padded[x*stride:x*stride+filter_size,
y*stride:y*stride+filter_size]

            # Performing element-wise multiplication between the
region and filter

            conv_output[x, y] = np.sum(region * filter_mat)

    return conv_output

```

```

# Performing Max-pooling Operation

def maxpool(input_mat, pool_size=2, stride=2):

    pool_output_dim = (input_mat.shape[0] - pool_size) // stride + 1

    pooled_output = np.zeros((pool_output_dim, pool_output_dim))

    for x in range(0, pool_output_dim):
        for y in range(0, pool_output_dim):
            # Selecting a pool_size x pool_size region

            pool_region = input_mat[x*stride:x*stride+pool_size,
y*stride:y*stride+pool_size]

            # Choosing the max value from the region

            pooled_output[x, y] = np.max(pool_region)

    return pooled_output

# Fully Connected Layer

def dense_layer(flat_input, weights, bias):

    return np.dot(weights, flat_input) + bias

# Defining CNN architecture

def cnn_model(input_img, conv_filters, dense_weights, dense_bias):

    # First convolution Layer with ReLU

```

```
conv1_output = convolution(input_img, conv_filters[0])

relu_output = relu_activation(conv1_output)


# First pooling layer

pooled_output = maxpool(relu_output)


# Flattening the Output for Fully Connected Layer

flattened_vector = pooled_output.flatten()


# Fully Connected Layer

dense_output = dense_layer(flattened_vector, dense_weights,
dense_bias)


# Softmax for Final Classification Probabilities

return softmax_activation(dense_output)


# Loading the Dataset

(train_images, train_labels), (test_images, test_labels) =
fetch_mnist()


# Defining sample filters, weights, and bias

conv_filters = [np.random.randn(3, 3)]

dense_weights = np.random.randn(10, 13 * 13)
```

```
dense_bias = np.random.randn(10)

# Model Testing

example_img = train_images[7]

# Forward passing through the CNN Model

predicted_output = cnn_model(example_img, conv_filters, dense_weights,
dense_bias)

# Displaying the Output Probabilities

print("Predicted Probabilities of The Model Are as Follows:")

for i in range(len(predicted_output)):

    print(f"Probability Image is of Class {i} Is:
{predicted_output[i]:.6f}")
```

Predicted Probabilities of The Model Are as Follows:

Probability Image is of Class 0 Is: 0.866019

Probability Image is of Class 1 Is: 0.000163

Probability Image is of Class 2 Is: 0.016885

Probability Image is of Class 3 Is: 0.011590

Probability Image is of Class 4 Is: 0.000001

Probability Image is of Class 5 Is: 0.000001

Probability Image is of Class 6 Is: 0.000005

Probability Image is of Class 7 Is: 0.000000

Probability Image is of Class 8 Is: 0.000013

Probability Image is of Class 9 Is: 0.105323